

AMASE: A Harness-Ablation Framework for Measuring Cost–Quality Trade-offs in Autonomous Software Engineering Agents

Aryan Raj Singh, <Team Member Name>, <Team Member Name>, <Team Member Name>

Department of Computer Science and Engineering

Noida Institute of Engineering and Technology, Greater Noida, Uttar Pradesh, India

Supervisor: Mr. <Supervisor Name>, Assistant Professor, CSE

Abstract—Autonomous software engineering agents couple a large language model to a development environment through a body of surrounding software known as the agent harness: the tool interface, control flow, context-management policy, loop-detection logic, model-selection strategy and instrumentation. Recent industrial evidence indicates that harness design alone accounts for approximately eleven to fifteen percentage points of task resolution with the underlying model held constant, an effect comparable to a full model generation. Despite this, the contribution of individual harness mechanisms has not been measured under controlled conditions, their token and monetary costs are almost never reported, and the benchmarks conventionally used for such comparison have been shown to be saturated and contaminated; SWE-bench Verified was formally retired by its principal industrial user in 2026 after an audit found 59.4% of sampled problems materially defective and documented verbatim reproduction of reference solutions by frontier models. This paper proposes AMASE, a harness-ablation framework that addresses these deficiencies jointly. AMASE implements a deliberately minimal ReAct agent in which eight harness mechanisms—environment context injection, pre-completion verification, doom-loop detection, context compaction, sub-agent delegation, retrieval mode, model routing and reasoning-budget allocation—are independently switchable experimental arms rather than architectural commitments. Every model call and tool invocation is instrumented to the OpenTelemetry GenAI semantic conventions, capturing tokens, monetary cost, latency percentiles and outcome. Each configuration is executed a minimum of three times so that variance may be reported, and scoring proceeds by end-state verification against both the public Terminal-Bench 2.x suite and a decontaminated held-out task set authored by the investigators. The intended contribution is an attribution table stating, for each mechanism, its marginal effect on resolution rate together with the cost of obtaining it, accompanied by a reproducible apparatus permitting the study to be extended.

Keywords—autonomous agents, large language models, agent harness, ablation study, software engineering benchmarks, LLM observability, model routing, benchmark contamination

I. INTRODUCTION

An autonomous software engineering agent operates a closed perception–action loop over a real development environment. It reads files, searches a repository, edits source, executes shell commands, runs a test suite, observes the resulting standard output and standard error, and revises its own work until a stated goal is satisfied or a resource budget is exhausted. Between 2023 and 2026 this class of system progressed from research prototype to routine industrial tooling.

As that transition occurred, the locus of engineering difficulty shifted. Obtaining a capable base model is no longer the constraint; constructing the software that surrounds it is. That software—the agent harness—comprises the tool interface exposed to the model, the system prompt and control flow, the strategy used to manage a finite context window, the mechanism that detects and escapes unproductive behaviour, the policy that determines which model serves which request, and the instrumentation that records what occurred. None of these components involves retraining. All of them materially alter system behaviour.

The magnitude of that alteration is the observation motivating this work. Two independent 2026 sources place harness design at eleven to fifteen percentage points of task resolution with the model fixed. In one published account a coding agent advanced from 52.8 to 66.5 on Terminal-Bench 2.0—a gain of 13.7 points, moving the system from outside the top thirty to the top five of the leaderboard—through harness modification alone.

Yet the harness remains largely unmeasured. Published agent research reports aggregate scores for complete systems, conflating the contribution of the model with that of the scaffolding. Individual mechanisms are described qualitatively and adopted on the strength of anecdote. Controlled evidence attributing a specific effect size to a specific mechanism is scarce, and evidence attaching a cost to that effect is scarcer still. A mechanism yielding two additional points at triple the token consumption is, for most deployments, a poor exchange; the present literature does not permit that judgement.

This paper proposes AMASE, a framework designed to supply exactly that evidence. Section II states the problem formally. Section III surveys related work. Section IV describes the proposed system. Section V presents the experimental design. Section VI states the expected results and Section VII concludes.

II. PROBLEM STATEMENT

Practitioners constructing autonomous software engineering agents must select among a growing catalogue of harness mechanisms. These selections are presently governed by anecdote, vendor documentation and imitation of published systems, because no controlled evidence exists

attributing a measured effect size or a monetary cost to any individual mechanism.

Three consequences follow. First, engineering effort is misallocated toward mechanisms that are fashionable rather than effective, while inexpensive high-yield mechanisms are neglected. Second, systems are deployed with unquantified unit economics, so that a technique improving resolution marginally at several times the token cost is adopted without that trade-off being visible to the deploying team. Third, the benchmarks ordinarily used to adjudicate such questions have lost validity.

The third consequence warrants elaboration, as it undermines the obvious remedy. In 2026 the principal industrial user of SWE-bench Verified withdrew it as a reported metric. An audit of 138 of its problems found 59.4% to contain material defects: 35.5% enforced implementation details so rigidly that functionally correct solutions were rejected, 18.8% tested behaviour absent from the problem statement, and 5.1% exhibited other faults. The same audit documented direct training contamination, with frontier models reproducing reference patches verbatim and recalling inline comments from the original source. A study evaluating solely against such an instrument inherits its invalidity.

The problem addressed by this work may therefore be stated precisely: there exists no reproducible, cost-annotated, contamination-resistant experimental framework capable of attributing the performance of an autonomous software engineering agent to its individual harness components. In consequence, the single most influential controllable factor in such systems remains, in the public literature, unquantified at the level of its constituent parts.

III. RELATED WORK

A. Agent Control Paradigms

ReAct [1] established the interleaving of explicit reasoning traces with environment actions that underlies most contemporary coding agents. Reflexion [2] added verbal self-reflection over failed attempts as a form of reinforcement without weight updates. Both define control structures; neither decomposes the surrounding machinery that renders those structures reliable in practice.

B. Coding Agents and Interfaces

SWE-agent [4] demonstrated that a purpose-built Agent-Computer Interface substantially raises resolution rates, establishing that interface design is consequential without identifying which interface elements carry the effect. OpenHands [5] provides an open platform for generalist software-development agents together with a shared evaluation suite, optimising a complete product in which individual scaffold mechanisms remain entangled.

C. Context and Cost

Liu et al. [6] demonstrate that retrieval accuracy degrades for information positioned toward the middle of a long context, diagnosing a failure mode without evaluating engineering remedies within an agent loop. RouteLLM [7] shows that learned routing between strong and weak models

preserves approximately 95% of quality at roughly 26% of cost, but evaluates on conversational benchmarks rather than long-horizon agentic software tasks. PagedAttention [8] addresses throughput at the serving layer and is orthogonal to harness-level decisions.

D. Benchmarks and Their Degradation

SWE-bench [3] established real GitHub issue resolution as the field standard before saturating and being retired [12]. SWE-rebench [9] supplies an automated pipeline for continuously refreshed, decontaminated tasks; SWE-Bench Pro [10] raises difficulty with 1,865 long-horizon tasks averaging 107 changed lines across 4.1 files; Terminal-Bench [11] contributes 89 Docker-sandboxed command-line tasks scored by end-state verification and, critically for this work, exposes an adapter interface permitting third-party agents to be evaluated. These contributions improve the quality of tasks; none addresses the attribution question.

E. Research Gap

The harness is established as the dominant controllable factor in coding-agent performance; its mechanisms have not been isolated; their costs are unreported; and the instruments used for such measurement are compromised. AMASE is constructed to satisfy all four conditions simultaneously.

IV. PROPOSED SYSTEM

A. Minimal Core and Switchable Middleware

The agent core is intentionally unsophisticated: a single ReAct loop implemented on a checkpointed state graph, exposing seven tools—bash, read, write, edit, grep, glob and run_tests. Checkpointing is required because controlled ablation depends on deterministic replay from an intermediate state. No middleware is active in the baseline; every result is reported as a delta from that baseline. Multi-agent decomposition is not assumed beneficial, but implemented as one switchable arm and tested against the single-agent control.

B. The Eight Ablation Arms

A1 injects a directory map, discovered tooling and an explicit turn and time budget at startup. A2 blocks the completion signal until the agent has verified its output against the specification and executed the tests. A3 tracks per-file edit counts and forces a re-plan on threshold. A4 selects among naive truncation, model summarisation and an external structured notes file. A5 executes review in a clean context window returning a condensed summary. A6 selects between agentic search over grep, glob and direct reads, and a hybrid pipeline combining BM25 sparse indexing, dense embeddings and cross-encoder reranking over AST-aware chunks. A7 applies static routing rules dispatching planning and debugging to a frontier model and mechanical edits to a locally hosted small model. A8 compares uniform reasoning effort against a high-plan, moderate-build, high-verify allocation.

C. Sandboxed Execution

Model-generated code is untrusted and never executes on the host. Each task runs in an ephemeral container with

networking disabled, provisioned at rollout start and destroyed on completion, with standard output and error returned to the agent. Ceilings on turn count, wall-clock duration and cumulative monetary spend are enforced at this boundary. A microVM implementation offering dedicated-kernel isolation is provided behind an identical interface for the multi-tenant threat model.

D. Instrumentation

Every model call and tool invocation emits a span conforming to the OpenTelemetry GenAI semantic conventions, exported to a self-hosted trace store. Conformance to an open standard rather than a vendor interface permits the backend to be replaced without reinstrumentation. Recorded quantities comprise resolution outcome, input and output tokens per model, monetary cost per model and per task, wall-clock duration at the fiftieth and ninety-fifth percentiles, turns to completion, tool-call error rate, doom-loop incidence and, for retrieval comparisons, tokens per task and time to first relevant file. Every reported figure is traceable to a stored span.

V. EXPERIMENTAL DESIGN

Configurations vary a single mechanism at a time against the baseline, followed by a reduced factorial over the three strongest mechanisms to detect interaction effects. Each configuration executes a minimum of three times, as single-run agent benchmark figures are dominated by stochastic variation; mean and spread are reported throughout. The model is held fixed within an experiment, with the full matrix executed against an inexpensive model and confirmatory runs against a frontier model. Hypotheses are pre-registered before execution.

Two task sets are used. The first is Terminal-Bench 2.x accessed through its adapter, which yields an externally comparable figure. The second comprises twenty-five to forty tasks authored by the investigators in the same format—task specification, container definition, reference solution and verification suite—drawn from repositories with no plausible training-corpus presence, and is therefore decontaminated by construction. Divergence between results on the two sets constitutes a finding in itself. Tasks resolved by the baseline in all repetitions or in none are discarded as carrying no discriminative signal.

Scoring proceeds by end-state verification: after the agent halts, the verification suite executes against the final environment state. Transcript grading is not employed. Model-based grading is confined to the single case in which no deterministic check exists, and its agreement with human judgement is reported. An automated regression suite executes on every change to prompts, middleware or configuration, failing the change if resolution falls below baseline beyond a defined tolerance.

VI. EXPECTED RESULTS

The primary artefact is an attribution table stating, for each of the eight mechanisms, its marginal contribution to resolution rate together with its token and monetary cost, reported with variance. Secondary artefacts comprise cost–

quality curves for the routing and retrieval arms with the operating knee identified, and a reproducible apparatus released alongside the decontaminated task set.

Negative findings are treated as results of equal standing. Establishing that a widely assumed technique fails to justify its cost on a defined task class is a substantive contribution to a literature currently dominated by untested assertion. It is specifically anticipated that hybrid retrieval may not outperform agentic search on repositories where exact symbol matching dominates, and that sub-agent delegation may prove counterproductive below a task-length threshold; both outcomes will be reported as measured.

VII. CONCLUSION

This paper has proposed AMASE, a harness-ablation framework for autonomous software engineering agents, motivated by the observation that the harness is worth an estimated eleven to fifteen percentage points of task resolution with the model held constant while remaining unmeasured at the level of its constituent mechanisms. The proposed system combines a deliberately minimal agent core, eight independently switchable middleware layers, comprehensive standards-conformant instrumentation, repetition with reported variance, an automated regression gate and evaluation against both a public benchmark and a contamination-free task set authored by the investigators. The work is narrow in scope and rigorous within it: it does not seek to compete with established agent platforms on capability, but to supply the controlled, cost-annotated evidence their construction currently lacks, and to demonstrate that disciplined empirical method remains applicable to systems whose behaviour is stochastic.

REFERENCES

- [1] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan and Y. Cao, “ReAct: Synergizing Reasoning and Acting in Language Models,” arXiv:2210.03629, 2022.
- [2] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan and S. Yao, “Reflexion: Language Agents with Verbal Reinforcement Learning,” arXiv:2303.11366, 2023.
- [3] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press and K. Narasimhan, “SWE-bench: Can Language Models Resolve Real-World GitHub Issues?,” arXiv:2310.06770, 2023.
- [4] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan and O. Press, “SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering,” in Proc. NeurIPS, 2024.
- [5] X. Wang, B. Li, Y. Song, F. F. Xu, X. Tang et al., “OpenHands: An Open Platform for AI Software Developers as Generalist Agents,” in Proc. ICLR, 2025.
- [6] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni and P. Liang, “Lost in the Middle: How Language Models Use Long Contexts,” TACL, vol. 12, 2024.
- [7] I. Ong, A. Almahairi, V. Wu, W.-L. Chiang, T. Wu, J. E. Gonzalez, M. W. Kadous and I. Stoica, “RouteLLM: Learning to Route LLMs with Preference Data,” arXiv:2406.18665, 2024.
- [8] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang and I. Stoica, “Efficient Memory Management for Large Language Model Serving with PagedAttention,” in Proc. ACM SOSP, 2023.

- [9] I. Badertdinov, A. Golubev, M. Nekrashevich et al., “SWE-rebench: An Automated Pipeline for Task Collection and Decontaminated Evaluation of Software Engineering Agents,” in Proc. NeurIPS, 2025.
- [10] Scale AI, “SWE-Bench Pro: Can AI Agents Solve Long-Horizon Software Engineering Tasks?,” arXiv:2509.16941, 2025.
- [11] M. A. Merrill, A. G. Shaw, N. Carlini, B. Li, H. Raj et al., “Terminal-Bench: Benchmarking Agents on Hard, Realistic Tasks in Command Line Interfaces,” arXiv:2601.11868, 2026.
- [12] OpenAI, “Why SWE-bench Verified No Longer Measures Frontier Coding Capabilities,” OpenAI Technical Report, 2026.
- [13] Anthropic, “Effective Context Engineering for AI Agents,” Anthropic Engineering, 2025.
- [14] LangChain, “Improving Deep Agents with Harness Engineering,” LangChain Engineering, 2026.
- [15] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu et al., “Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena,” in Proc. NeurIPS, 2023.